



Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Prefix

Infix : $\overset{(1)}{a} \overset{(2)}{*} \overset{(3)}{b} / \overset{(5)}{c} \overset{(6)}{+} \overset{(7)}{e} - \overset{(4)}{f} \overset{(7)}{*} h + l$
 $\underset{\text{Infix}}{*} a b / c \overset{\text{Infix}}{*} d + e - f \overset{\text{Infix}}{*} h + l$
 $\underset{\text{Infix}}{/} \overset{\text{Infix}}{*} a b c \overset{\text{Infix}}{*} d + e - f \overset{\text{Infix}}{*} h + l$
 $\underset{\text{Infix}}{*} / \overset{\text{Infix}}{*} a b c d + e - f \overset{\text{Infix}}{*} h + l$
 $\underset{\text{Infix}}{*} / \overset{\text{Infix}}{*} a b c d + e - \overset{\text{Infix}}{*} f h + l$
 $\underset{\text{Infix}}{+} \overset{\text{Infix}}{*} / \overset{\text{Infix}}{*} a b c d e - \overset{\text{Infix}}{*} f h + l$
 $\underset{\text{Infix}}{-} \underset{\text{Infix}}{+} \overset{\text{Infix}}{*} / \overset{\text{Infix}}{*} a b c d e \overset{\text{Infix}}{*} f h + l$
 $\underset{\text{Infix}}{+} - \underset{\text{Infix}}{+} \overset{\text{Infix}}{*} / \overset{\text{Infix}}{*} a b c d e \overset{\text{Infix}}{*} f h + l$

Postfix

Infix : $\overset{(1)}{a} \overset{(2)}{*} \overset{(3)}{b} / \overset{(5)}{c} \overset{(6)}{+} \overset{(7)}{e} - \overset{(4)}{f} \overset{(7)}{*} h + l$
 $\underset{\text{Infix}}{a b} \overset{\text{Infix}}{*} / c \overset{\text{Infix}}{*} d + e - f \overset{\text{Infix}}{*} h + l$
 $\underset{\text{Infix}}{a b} \overset{\text{Infix}}{*} c / \overset{\text{Infix}}{*} d + e - f \overset{\text{Infix}}{*} h + l$
 $\underset{\text{Infix}}{a b} \overset{\text{Infix}}{*} c / d \overset{\text{Infix}}{*} + e - f \overset{\text{Infix}}{*} h + l$
 $\underset{\text{Infix}}{a b} \overset{\text{Infix}}{*} c / d \overset{\text{Infix}}{*} + e - \overset{\text{Infix}}{*} f h + l$
 $\underset{\text{Infix}}{a b} \overset{\text{Infix}}{*} c / d \overset{\text{Infix}}{*} e + - \overset{\text{Infix}}{*} f h + l$
 $\underset{\text{Infix}}{a b} \overset{\text{Infix}}{*} c / d \overset{\text{Infix}}{*} e + f h \overset{\text{Infix}}{*} - + l$
 $\underset{\text{Infix}}{a b} \overset{\text{Infix}}{*} c / d \overset{\text{Infix}}{*} e + f h \overset{\text{Infix}}{*} - l +$

Ascending stack

$top = -1$

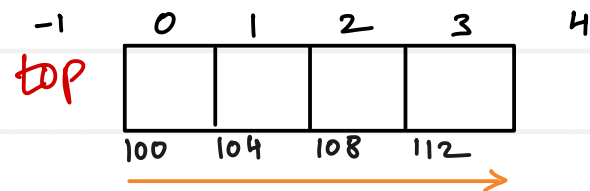
push : $top++$
 $arr[top] = value$

pop : $top--$

peek : $arr[top]$

Empty : $top == -1$

Full : $top == size-1$



Descending stack

$top = size$

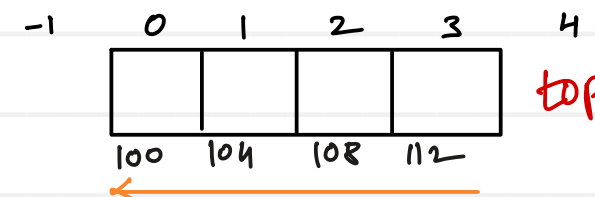
push : $top--$
 $arr[top] = value$

pop : $top++$

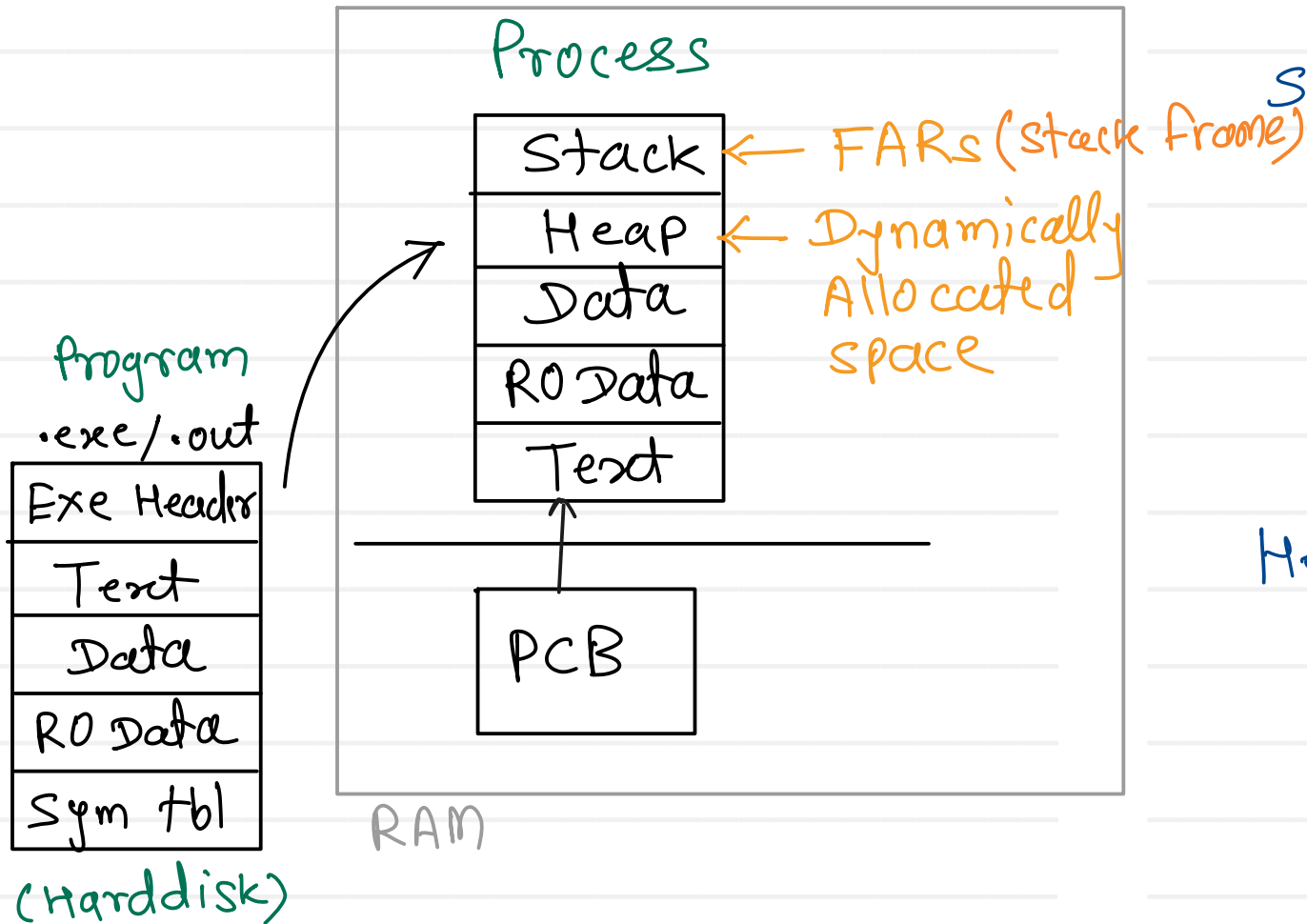
peek : $arr[top]$

Empty : $top == size$

Full : $top == 0$



OS Interview



DS Interviews

Stack :

- utility DS
- LIFO behaviour
- operations - push/pop/peek
- can be implemented using array or linked list.

Heap :

- hierarchical DS
- array implementation of Complete Binary Tree.
- max heap, min heap

Stack

- 1> Parenthesis balancing [Lexical analysis]
- 2> Expression conversion and evaluation
- 3> Function calls / recursion
- h> Used in advanced data structures for traversing

- **Expression conversion and evaluation:**

- Infix to postfix
- Infix to prefix
- Postfix evaluation
- Prefix evaluation

Queue

- 1> Jobs submitted to printer [spooler directory]
- 2> In Network setups – file access of file server machine is given to First come First serve basis
- 3> Calls are placed on a queue when all operators are busy
- h> Used in advanced data structures to give efficiency.
- 5> Process waiting queues in OS
- 6> CPU scheduling in OS (ready queue)
- 7> Waiting Lines (real life)
- 8> token Systems (banks, food plaza)
- 9> Data buffers

Stack

- 5) Undo/Redo operations
- 6) Backtracking algorithms
- 7) Reversal of data
- 8) Web browser's history
- 9) Operating systems (Int Handling/
Context switching)

Queue

- 10) Message queues
- 11) Web server request handling
- 12) Traffic system
- 13) resource sharing
- 14) multi player games
- 15) music players (play list)

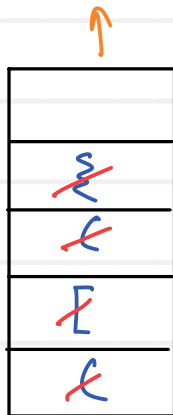
Parenthesis balancing using stack

1. check from left to right
2. if opening parenthesis
push it on stack.
3. if closing parenthesis
 - 3.1 if stack is empty, return false
 - 3.2 pop one parenthesis from stack and compare with current parenthesis
if matching, continue
if not matching, then return false
4. if expression is finished & stack is not empty
return false.
5. if passes all above steps, then return true

Parenthesis balancing using stack

$5 + ([9 - 4] * (8 - \{6 / 2\}))$

$] == [$
 $\} == \{$
 $) == ($
 $) == ($



stack

$5 + ([9 - 4] * (8 - \{6 / 2\}))$

$] == [$
 $\} == \{$
 $] \neq ($



stack

opening

(	[	{
0	1	2

closing

)	]	}
0	1	2

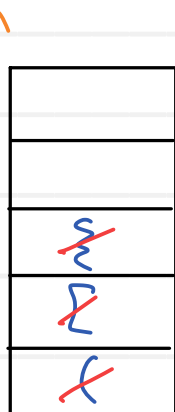
String

↓
indexOf()

↓
returns index of char
returns -1 if char
not found

$5 + ([9 - 4] * 8 - \{6 / 2\})$

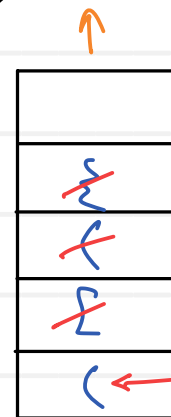
$] == [$
 $\} == \{$
 $) == ($
 $) == ?$



stack

$5 + ([9 - 4] * (8 - \{6 / 2\}))$

$] == [$
 $\} == \{$
 $) == ($



stack

Valid Parantheses

Given a string s containing just the characters '(', ')', '{', '}', '[' and ''], determine if the input string is valid.

An input string is valid if:

- Open brackets must be closed by the same type of brackets.
- Open brackets must be closed in the correct order.
- Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: s = "()"

Output: true

Example 2:

Input: s = "()[]{}"

Output: true

Example 3:

Input: s = "("

Output: false

Example 4:

Input: s = "([)"

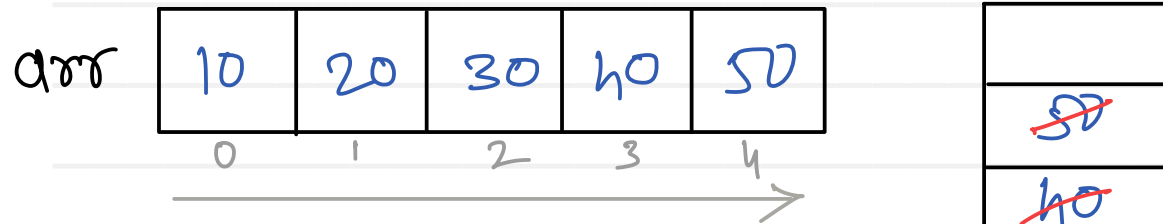
Output: true

```
boolean isValid(String s) {  
    Stack<Character> st = new Stack<>();  
    for (i = 0; i < s.length(); i++) {  
        char ele = s.charAt(i);  
        if (ele == '(' || ele == '[' || ele == '{')  
            st.push(ele);  
        else if (ele == ')' && !st.isEmpty() && st.peek() == '(')  
            st.pop();  
        else if (ele == ']' && !st.isEmpty() && st.peek() == '[')  
            st.pop();  
        else if (ele == '}' && !st.isEmpty() && st.peek() == '{')  
            st.pop();  
        else  
            return false;  
    }  
    if (!st.isEmpty())  
        return false;  
    return true;  
}
```

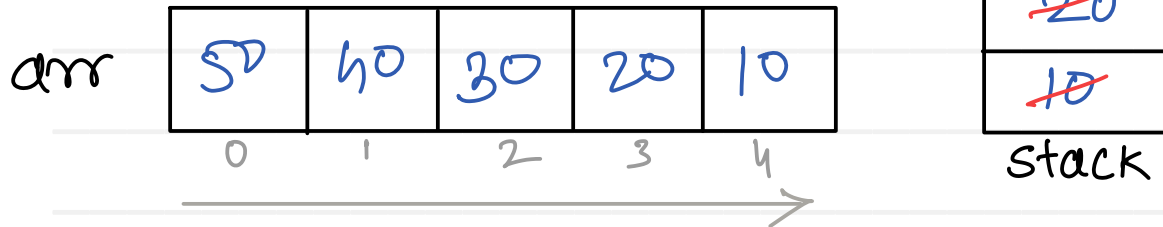
Reverse array, string or linked list using stack/queue

`int arr[] = {10, 20, 30, 40, 50}`

Before



After



```
void reverseArray(int arr[]) {
    Stack<Integer> st = new Stack<>();
    for(int i = 0; i < arr.length; i++)
        st.push(arr[i]);
    for(int i = 0; i < arr.length; i++)
        arr[i] = st.pop();
}
```

Time complexity:

Total itrs = $n + n = 2n$

$T \propto 2n$

$T(n) = O(n)$

Space Complexity:

Auxiliary space = n (stack space)

$S(n) = O(n)$

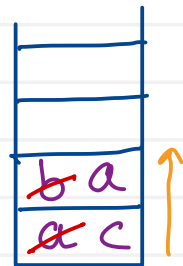
Remove all adjacent duplicates in string

You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them.

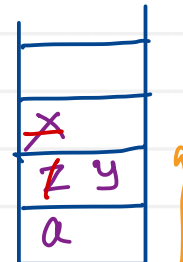
We repeatedly make duplicate removals on s until we no longer can.

Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique.

Example 1:
Input: $s = \text{"abbaca"}$
Output: "ca"



Example 2:
Input: $s = \text{"azxxzy"}$
Output: "ay"



```
String removeDuplicates(String s) {
    int n = s.length();
    char[] st = new char[n];
    int top = -1;
    for (int i = 0; i < n; i++) {
        char ch = s.charAt(i);
        if (top > -1 && ch == st[top])
            top--;
        else {
            top++;
            st[top] = ch;
        }
    }
    return new String(st, 0, top + 1);
}
```

Auxiliary space

$T(n) = O(n)$
 $S(n) = O(n)$

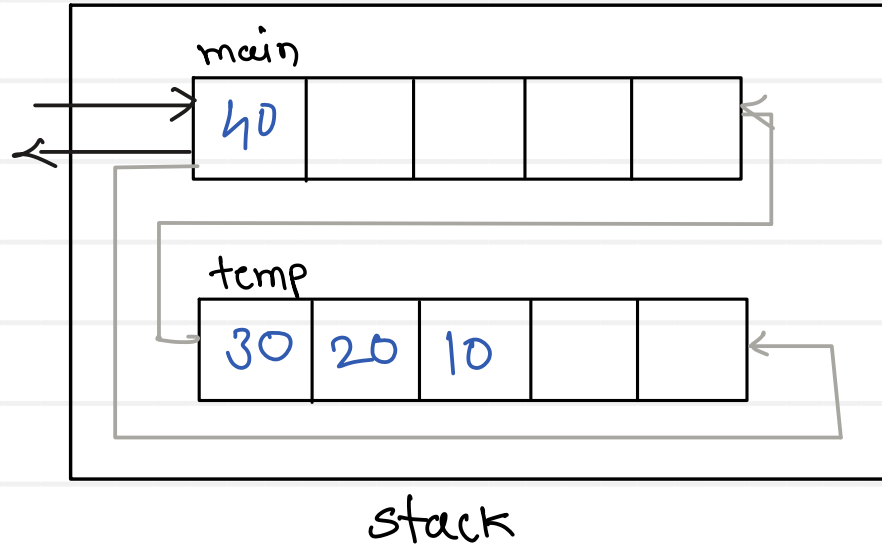
array starting index length of string

Create stack using queue

LIFO

FIFO

Push order : 10, 20, 30, 40



Pop order : 40, 30, 20, 10

push :

```
while (!mq.isEmpty())
    tq.offer(mq.poll());
mq.offer(value);
while (!tq.isEmpty())
    mq.offer(tq.poll());
```

pop :

```
mq.poll()
```

peek :

```
mq.peek()
```

Push :

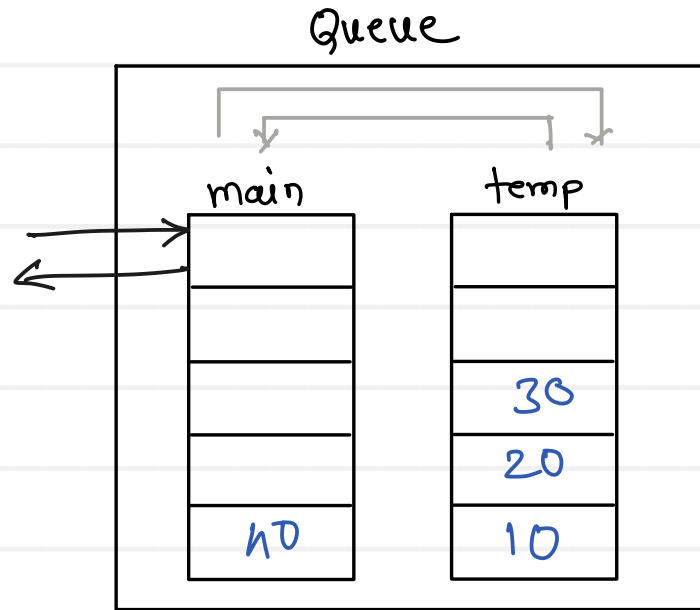
itrs = $n + n$
Time $\propto 2n$
 $T(n) = O(n)$

Pop & Peek :

$T(n) = O(1)$

Create queue using stack

Push order : 10, 20, 30, 40



Pop order : 10, 20, 30, 40

offer :

```
while (!ms.isEmpty())
    ts.push(ms.pop());
ms.push(value);
while (!ts.isEmpty())
    ms.push(ts.pop());
```

poll :

```
ms.pop()
```

peek :

```
ms.peek()
```

Push :

itrs = n + n

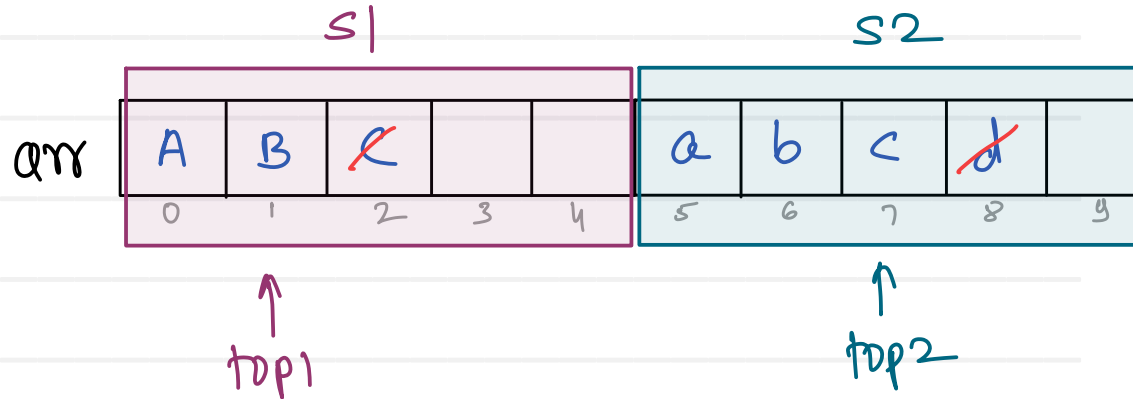
Time $\propto 2n$

$T(n) = O(n)$

Pop & Peek :

$T(n) = O(1)$

How to Implement two stacks using array efficiently?



← both stacks (S1 & S2) will be with fixed capacity

$top1 = -1$

Empty : $top1 == -1$

full : $top1 == length/2$

$top2 = length/2$

Empty : $top2 == length/2$

full : $top2 == length-1$

Stack 1

init : $top1 = -1$

push : $top1++$

$arr[top1] = val$

pop : $top1--$

isEmpty : $top1 == -1$

isfull : $top1 == top2-1$

Stack 2

init : $top2 = length$

push : $top--$

$arr[top] = val$

pop : $top++$

isEmpty : $top2 == length$

isfull : $top1 == top2-1$

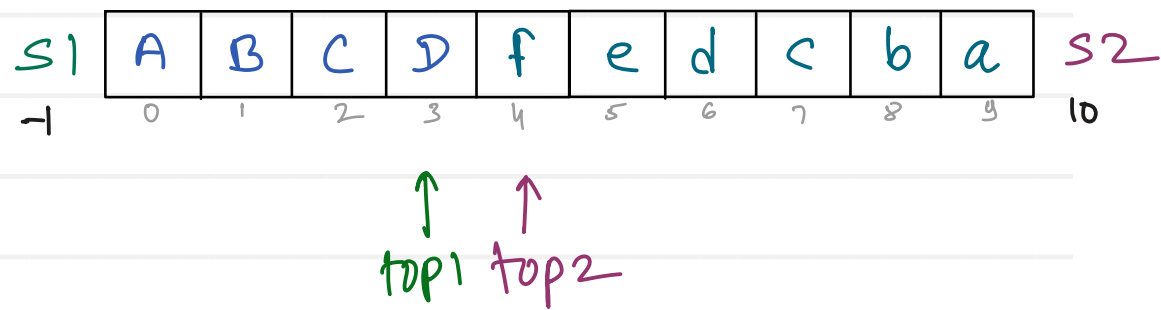
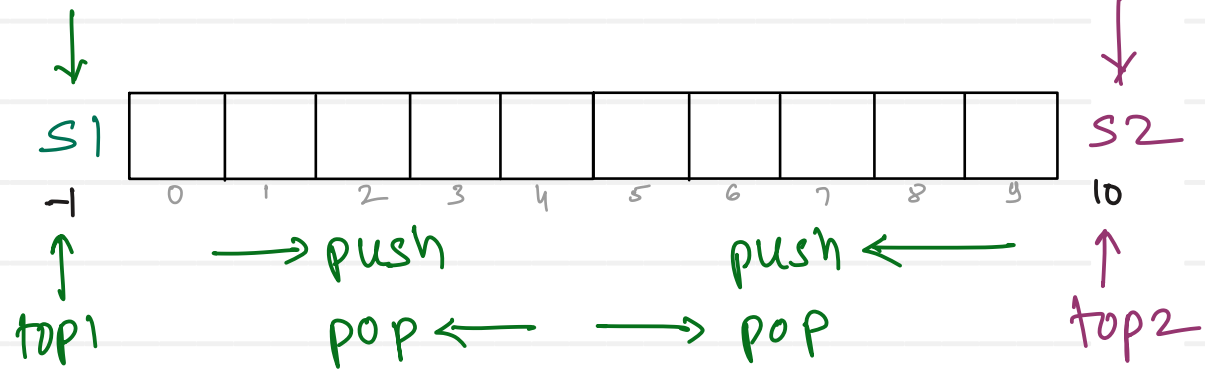
How to Implement two stacks using array efficiently?

Ascending
stack

Descending
stack

← both stacks (s1 & s2)
will be with variable capacity

- more memory efficient



Rat in a maze problem

Algorithm:

1. push src cell on stack & mark it as visited
2. While stack is not empty:
 - 2.1 pop a cell from stack (curr)
 - 2.2 if (curr cell == dest cell)

return true
 - 2.3 pull all possible/valid unmarked neighbor cells on stack
3. if destination is not reachable

return false.

hh
 3h
 33
 23
 1h
 13
 02
 12
 30
 h2
 h1
 31
 21
 11
 10
 00

stack

■ - 0 (blocked)
 ■ - 1 (open)

maze

	0	1	2	3	4
0	S 00	01	02	03	04
1	10	11	12	13	14
2	20	21	22	23	24
3	30	31	32	33	34
4	40	41	42	43	d 44

marked →
 (true/false)
 ↑ ↑
 visited not
 visited

	0	1	2	3	4
0	00	01	02	03	04
1	10	11	12	13	14
2	20	21	22	23	24
3	30	31	32	33	34
4	40	41	42	43	44